COSC 416 April 2026

Okanagan College, Kelowna (BC) Canada.

SciKey
Transition & Deployment

Project Sponsor: Gaetan Hains

Université Paris-Est Créteil

John Cortez, Brandon Hay &

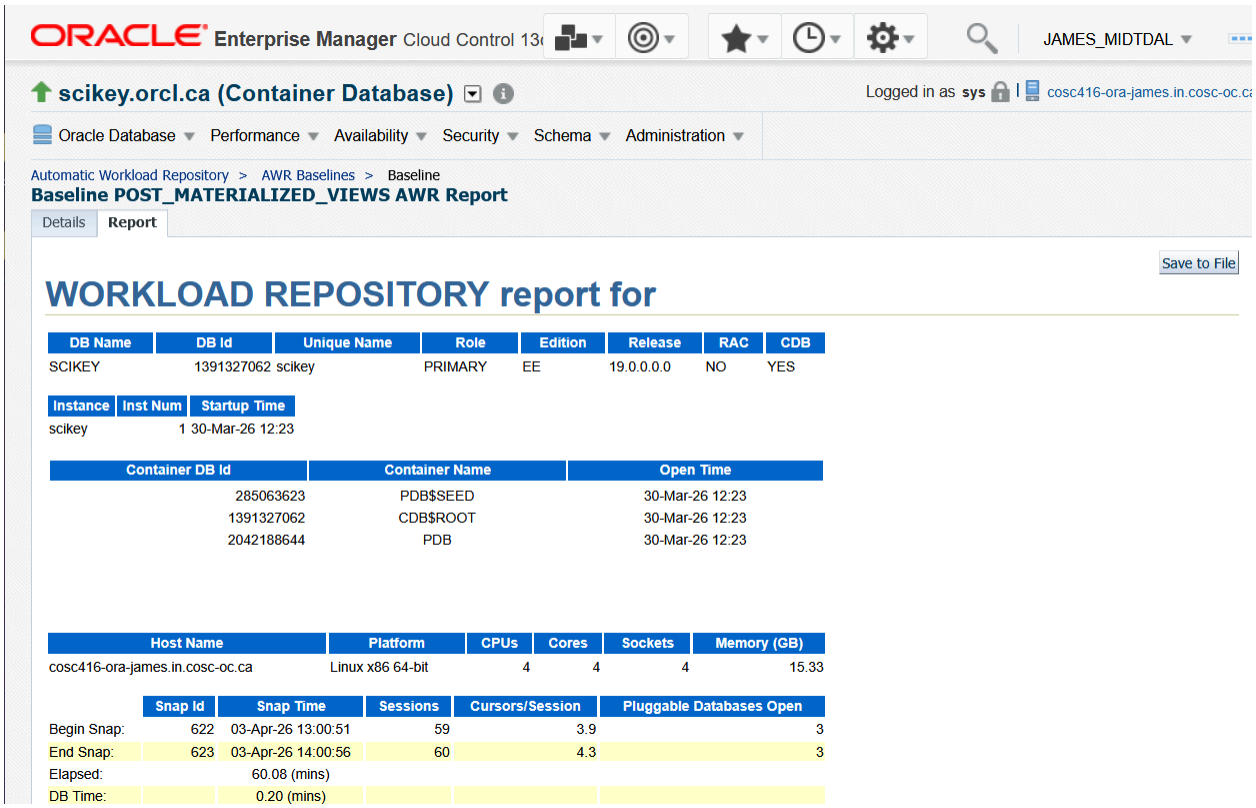
James Midtdal

Table Of Contents

Baseline Performance Review 2
Capacity & Scalability Plan 5
Operational Runbooks 6
Knowledge Transfer 7
Version Control - GitHub Repository 7
Automated Performance Testing 9
Blue-Green Deployment with Gradual Traffic Migration 10
Post-Deployment Monitoring 11

Baseline Performance Review

After tuning our instance and operating system, and optimizing tables through indexes, materialized views, and partitions, we can observe the resulting healthy baseline:



DB Name	DB Id	Unique Name	Role	Edition	Release	RAC	CDB
SCIKEY	1391327062	scikey	PRIMARY	EE	19.0.0.0.0	NO	YES

Instance	Inst Num	Startup Time
scikey	1	30-Mar-26 12:23

Container DB Id	Container Name	Open Time
285063623	PDB$SEED	30-Mar-26 12:23
1391327062	CDB$ROOT	30-Mar-26 12:23
2042188644	PDB	30-Mar-26 12:23

Host Name	Platform	CPUs	Cores	Sockets	Memory (GB)
cosc416-ora-james.in.cosc-oc.ca	Linux x86 64-bit	4	4	4	15.33

	Snap Id	Snap Time	Sessions	Cursors/Session	Pluggable Databases Open
Begin Snap:	622	03-Apr-26 13:00:51	59	3.9	3
End Snap:	623	03-Apr-26 14:00:56	60	4.3	3
Elapsed:		60.08 (mins)			
DB Time:		0.20 (mins)			

Figure 1.

Report Summary

Load Profile

	Per Second	Per Transaction	Per Exec	Per Call
DB Time(s):	0.0	0.1	0.00	0.00
DB CPU(s):	0.0	0.1	0.00	0.00
Background CPU(s):	0.0	0.5	0.00	0.00
Redo size (bytes):	2,697.6	85,303.7		
Logical read (blocks):	71.2	2,250.3		
Block changes:	12.0	379.7		
Physical read (blocks):	0.0	0.6		
Physical write (blocks):	0.6	17.4		
Read IO requests:	0.0	0.6		
Write IO requests:	0.3	10.4		
Read IO (MB):	0.0	0.0		
Write IO (MB):	0.0	0.3		
IM scan rows:	0.0	0.0		
Session Logical Read IM:	0.0	0.0		
User calls:	1.4	44.5		
Parses (SQL):	4.1	129.0		
Hard parses (SQL):	0.3	9.5		
SQL Work Area (MB):	0.1	4.3		
Logons:	0.1	1.9		
User logons:	0.0	0.1		
Executes (SQL):	6.2	196.5		
Rollbacks:	0.0	0.1		
Transactions:	0.0			

Instance Efficiency Percentages (Target 100%)

Buffer Nowait %:	100.00	Redo NoWait %:	100.00
Buffer Hit %:	99.97	In-memory Sort %:	100.00
Library Hit %:	92.24	Soft Parse %:	92.65
Execute to Parse %:	34.35	Latch Hit %:	99.87
Parse CPU to Parse Elapsd %:	92.33	% Non-Parse CPU:	96.19
Flash Cache Hit %:	0.00		

Figure 2.

Some notable statistics are as follows. In-Memory Sort is at 100%, showing we are never spilling to disk and our PGA is appropriately sized. Buffer Nowait is at 100%, showing zero contention for blocks in the buffer cache. Redo Nowait is at 100%, demonstrating that our log buffer is appropriately sized. Buffer Hit is at 99.97%, indicating that nearly all data requested by our queries is found in the buffer cache and virtually none needs to be loaded from disk.

An area for improvement is the Soft Parse ratio at 92.65%; ideally this would be above 95%. The main issue is that our stress test opens and closes cursors every iteration, causing extra parse calls. The query could be rewritten, or SESSION_CACHED_CURSORS could be enabled. Either change should also help with the low Execute to Parse ratio.

Top 10 Foreground Events by Total Wait Time

Event	Waits	Total Wait Time (sec)	Avg Wait	% DB time	Wait Class
DB CPU		12.7		104.7	
log file sync	33	.3	8.73ms	2.4	Commit
Disk file operations I/O	2,548	.2	63.44us	1.3	User I/O
PX Deq: Slave Session Stats	282	.1	418.07us	1.0	Other
PX Deq: Join ACK	120	.1	786.21us	.8	Other
db file sequential read	49	.1	1.37ms	.6	User I/O
direct path write	12	0	4.15ms	.4	User I/O
cursor: pin S wait on X	12	0	4.05ms	.4	Concurrency
enq: PS - contention	25	0	1.82ms	.4	Other
PGA memory operation	2,711	0	15.41us	.3	Other

Wait Classes by Total Wait Time

Wait Class	Waits	Total Wait Time (sec)	Avg Wait Time	% DB time	Avg Active Sessions
System I/O	20,342	25	1.22ms	203.9	0.0
DB CPU		13		104.7	0.0
Other	5,506	2	395.70us	17.9	0.0
Commit	49	0	8.96ms	3.6	0.0
User I/O	3,828	0	90.13us	2.8	0.0
Concurrency	69	0	1.24ms	.7	0.0
Network	2,558	0	4.45us	.1	0.0
Application	19	0	274.79us	.0	0.0

Figure 3.

Our top wait events show that the system is mostly idle, with log file sync accounting for a very minimal 2.4% of DB CPU time. All other wait events are minimal, indicating a clean, healthy database with no major issues.

Capacity & Scalability Plan

With materialized views now handling much of the heavy joins, we can attempt to predict the next most likely failures. The following figure shows a report after running the stress test using materialized views and serves as the extrapolation point for failure.

Top 10 Foreground Events by Total Wait Time

Event	Waits	Total Wait Time (sec)	Avg Wait	% DB time	Wait Class
resmgr:cpu quantum	5,052	2998.4	593.50ms	69.9	Scheduler
DB CPU		341.8		8.0	
log file sync	742	166.5	224.36ms	3.9	Commit
cursor: pin S	3,586	64.5	17.99ms	1.5	Concurrency
latch free	734	51.9	70.70ms	1.2	Other
cursor: pin S wait on X	39	24.3	622.11ms	.6	Concurrency
library cache lock	177	20.3	114.64ms	.5	Concurrency
cursor: mutex X	58,200	19.1	327.66us	.4	Concurrency
cursor: mutex S	270	14.6	54.16ms	.3	Concurrency
row cache mutex	75	3.5	46.50ms	.1	Concurrency

Wait Classes by Total Wait Time

Wait Class	Waits	Total Wait Time (sec)	Avg Wait Time	% DB time	Avg Active Sessions
Scheduler	5,052	2,998	593.50ms	69.9	14.1
DB CPU		342		8.0	1.6
Commit	742	166	224.36ms	3.9	0.8
Concurrency	62,587	149	2.38ms	3.5	0.7
Other	882	52	58.84ms	1.2	0.2
Network	16,265	7	442.53us	.2	0.0
System I/O	226	0	491.32us	.0	0.0
User I/O	65	0	1.29ms	.0	0.0

Figure 4.

The biggest bottleneck is already the Resource Manager, and it will continue to get worse if the number of connections or query complexity increases. Another important bottleneck is the log file sync; an increase in commits per session will only make this bottleneck worse. Finally, to hit a meaningful CPU bottleneck, the amount of work done per session would have to scale to approximately 2.5 times the current stress test, assuming the other bottlenecks are resolved first. The justification for 2.5 times is that the current workload consumes around 40% (1.6 average active sessions of a possible 4 on a 4-core CPU) of total available DB CPU, leaving enough headroom to almost triple the workload before fully saturating all 4 cores.

The current dataset is approximately 3.07 GB, and the current total instance memory allocation is 9.2 GB. With the materialized views in place, even doubling the dataset puts the system at risk of starting to rely on the I/O subsystem. The current stress tests use filters on queries, so there is a risk of CPU bottlenecks if unselective aggregations are run across the materialized views.

Operational Runbooks

During the construction phase, performance bottlenecks were identified through stress testing and AWR analysis, including high CPU usage, low connection success rates, and excessive redo log churn .

The following runbooks provide actionable steps to address these issues:

- **High CPU / resmgr:cpu quantum wait**
  - Cause: CPU saturation under high concurrency
  - Action: Reduce concurrent sessions or optimize query execution
  - Example: High DB CPU time observed during stress testing
- **Low Connection Success Rate**
  - Cause: Too many concurrent connections overwhelming the database
  - Action: Enable Database Resident Connection Pooling (DRCP)
  - Result: Improved from ~1400 to ~2900 successful connections
- **Log file sync wait / Redo Log Churn**
  - Cause: Frequent commits and small redo logs
  - Action: Increase redo log size and reduce commit frequency
  - Result: Improved log turnover from ~90 seconds to significantly longer intervals
- **Full Table Scans / Inefficient Queries**
  - Cause: Missing indexes or poor execution plans
  - Action: Add indexes on frequently filtered columns and re-evaluate execution plans

These runbooks provide clear, repeatable steps for resolving performance issues observed during system construction.

Knowledge Transfer

From our stress test, the following eight queries were selected as the most useful. All are performing work using materialized views, which should be prioritized for performance:

- Query 4: Documents with keywords
- Query 5: Documents with authors
- Query 6: Author affiliations joined with document_author_keyword
- Query 7: Full star join
- Query 8: Authors by document count
- Query 9: Organisms by distinct author count
- Query 11: Filtered star join alternate keyword via MV
- Query 12: Multi-affiliation authors

After examining the Explain Plans, the following results were observed:

Queries 4, 7, and 11 all use the index `IDX_MV_DAK_KEYWORD_TEXT` with an index range scan and minimal I/O.

Query 5 uses `IDX_MV_DAK_LAST_NAME` and an index range scan. Broader prefixes in the author name result in more ROWID lookups back to the materialized view.

Query 6 results in both materialized views (`MV_AUTHOR_ORGANISM_DISTINCT` and `MV_DOC_AUTHOR_KEYWORD`) being full-table-scanned. This is a good area to focus on for further tuning. Recommendation: create index `IDX_MV_DAK_AUTHOR_KEY`.

Query 8 performs a full table scan on `MV_AUTHOR_DOC_COUNTS` with a filter on `last_name`, as the materialized view is quite small (443 rows). If the materialized view grows significantly, an index on `last_name` would be beneficial.

Query 9 performs a full table scan of `MV_AUTHOR_ORGANISM_DISTINCT`. This result is expected due to the lack of filtering. No indexing or partitioning strategy will help here.

Query 12 results in a full scan of `MV_AUTHOR_ORGANISM_DISTINCT` with a double hash group-by. The filter reduced the number of rows to approximately 3,133, but an index on `last_name` should be considered if performance dips.

Version Control - GitHub Repository

The database setup follows a simple directory structure, separating database concerns into discrete files by responsibility. `TableSpace.sql` defines the physical storage layer and must be executed first, as both `PartitionedTables.sql` and `AdvancedIndexes.sql` depend on the `scikey_data` tablespace existing before any table or index DDL can resolve.

The data insertion layer is located in the same directory — `dataLoader.py` handles population from the HAL archive using the structure defined by `sample_json_record.json` as a reference document for the expected input shape. Keeping the loader alongside the schema means any DDL change that affects how records are ingested — a new column on `document`, a change to how `keyword_text` is stored — is visible in the same repository diff as the Python code that writes those records, making the impact of a schema change easier to reason about during review.

`AdvancedIndexes.sql` is executed last, as indexes reference tables that must already exist and should be added after data is inserted for efficiency. Finally the `README.md` serves as the guide for the execution sequence.

This simple structure is all that's required for this database as there is no need to overcomplicate it.

The public repo can be found here: https://github.com/ToxicOstrich/OC_SciKey_COSC_416

Automated Performance Testing

To maintain performance stability, performance testing is incorporated into the deployment process to detect any degradation caused by schema changes.

A set of representative workload queries, based on the system's core operations such as joins, aggregations, and materialized view access, are executed before and after any modifications to the database.

The execution time of these queries is compared against a baseline established during the construction and stress testing phase.

If the execution time of any query increases by more than **10%**, the deployment is rejected to prevent performance regression.

Process:

1. Execute baseline workload queries
2. Apply schema changes (indexes, partitions, materialized views)
3. Re-run the same queries
4. Compare execution times
5. Reject deployment if performance degradation exceeds the threshold

This ensures that all changes introduced to the system maintain or improve performance and do not negatively impact query execution.

Blue-Green Deployment with Gradual Traffic Migration

Blue-green deployment is a release strategy where you can maintain two functionally identical database environments. The Blue represents the current production instance and Green represents the staged environment receiving schema changes. This deployment solution enables zero-downtime schema changes with an immediate rollback capability should degradation occur.

Rather than switching all traffic instantaneously from Blue to Green, production load is migrated incrementally through a series of waves managed at the load balancer or connection pool layer. In the initial wave, Green will receive only a small fraction of traffic (5%) and critically, this early exposure is restricted to read operations. This strategy should not be done when new document writes are scheduled to avoid inconsistent database instance states.

Once read traffic on Green demonstrates stable query plans and acceptable AWR metrics relative to the Blue baseline, the traffic load shift progresses through subsequent waves. These waves will gradually increase in value such as 25%, 50%, 75% database traffic being routed to the Green instance. The 50% threshold is a particularly important checkpoint, as it is the first point at which both environments are absorbing equivalent read load and their performance can be compared directly under identical conditions. At any point if there are errors, unfavorable performance, or acceptable baseline thresholds being crossed. The system can alert the load balancer and route the traffic back to the safe Blue instance and log the errors with the Green instance for dba review.

Upon successful completion of Wave 5, Blue is retained in a warm standby state for a defined period to ensure no issues spring up upon the full workload being deployed on Green. If the period has ended without errors, Blue is decommissioned and recycled as the next Green environment for future deployments.

Post-Deployment Monitoring

After deployment, both real-time monitoring and historical analysis are used to ensure that no performance regression is introduced.

Real-time monitoring is performed using Oracle Enterprise Manager (OEM), where key performance indicators such as CPU usage, wait events, and active sessions are continuously observed. This allows immediate detection of abnormal spikes in system activity.

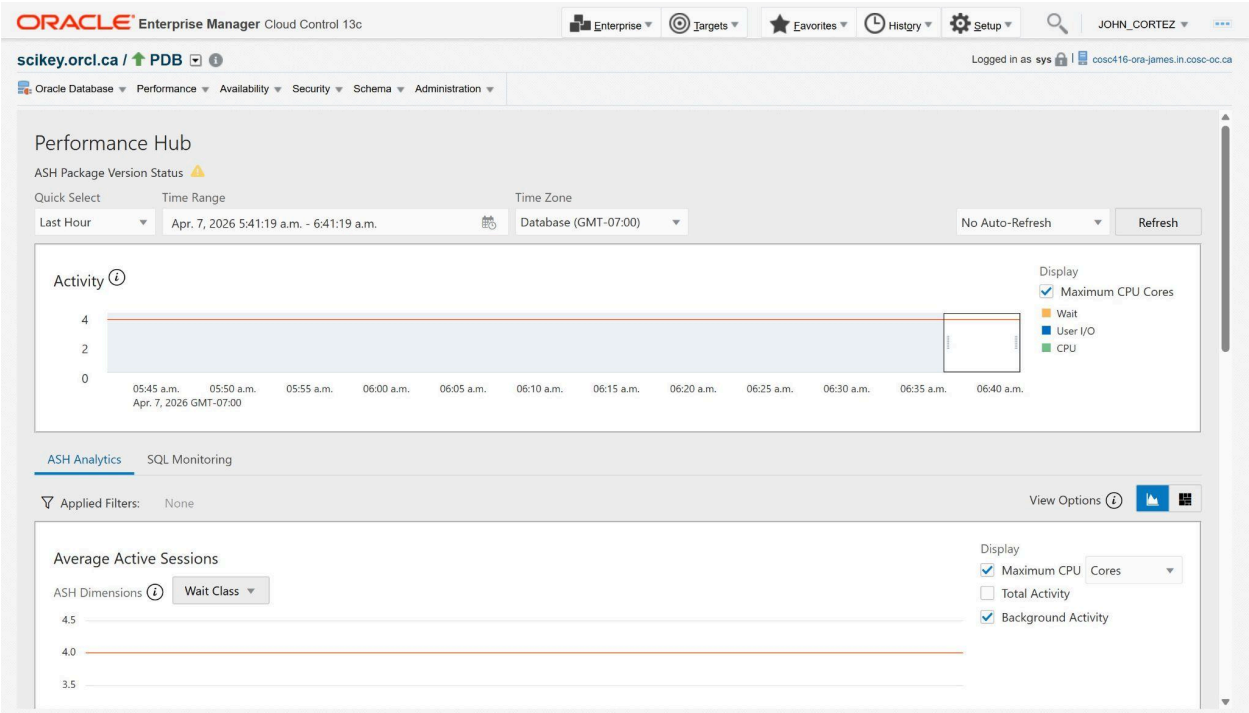


Figure 5.

The Activity graph displays the distribution of database workload across CPU and wait classes, allowing identification of bottlenecks such as CPU contention or I/O delays.

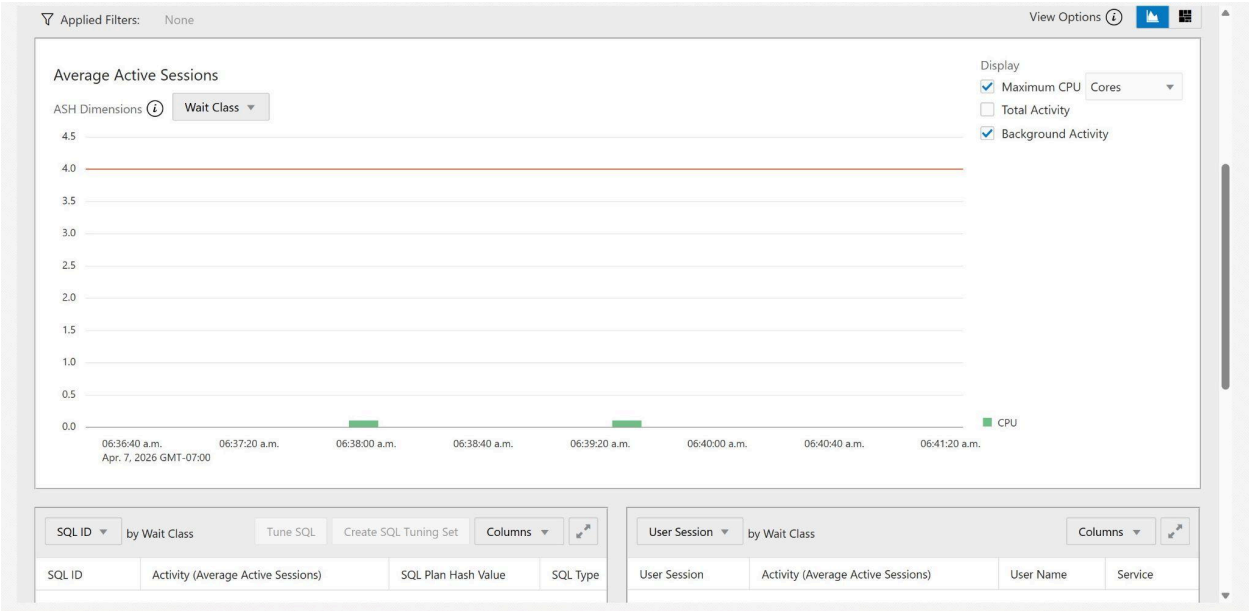
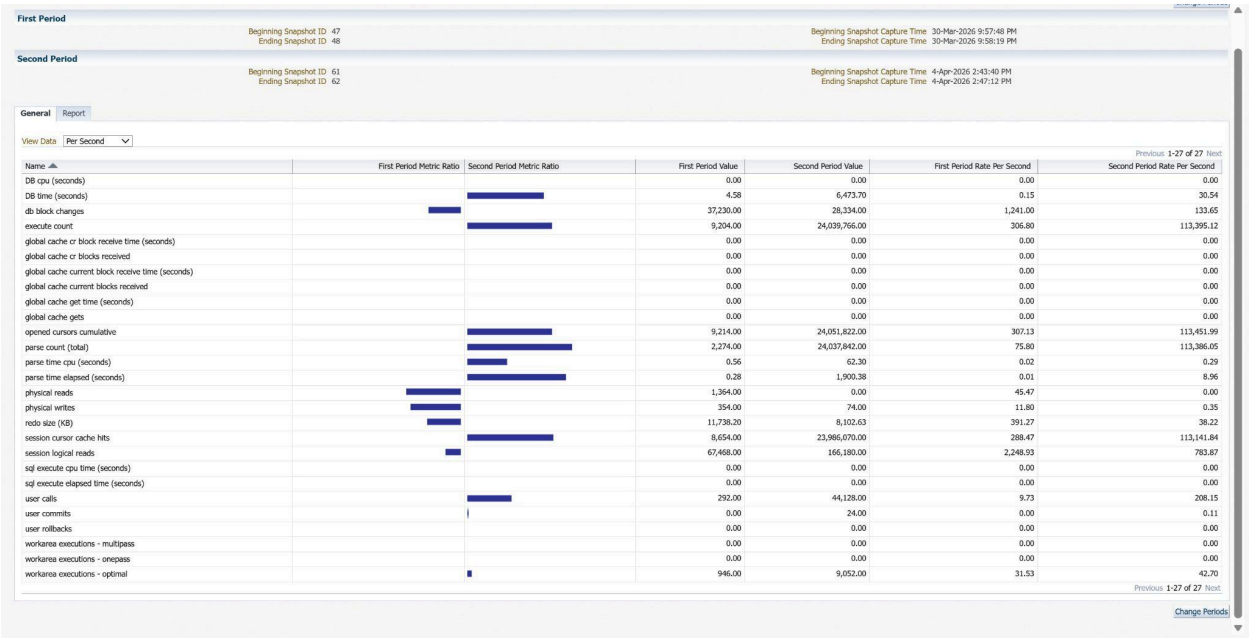


Figure 6.

In addition, Automatic Workload Repository (AWR) snapshots are used to compare current performance against previous system baselines.



First Period

Beginning Snapshot ID 47
Ending Snapshot ID 48
Beginning Snapshot Capture Time 30-Mar-2026 9:57:48 PM
Ending Snapshot Capture Time 30-Mar-2026 9:58:19 PM

Second Period

Beginning Snapshot ID 61
Ending Snapshot ID 62
Beginning Snapshot Capture Time 4-Apr-2026 2:43:40 PM
Ending Snapshot Capture Time 4-Apr-2026 2:47:12 PM

General | Report

View Data: Per Second

Previous 1-27 of 27 Next

Name	First Period Metric Ratio	Second Period Metric Ratio	First Period Value	Second Period Value	First Period Rate Per Second	Second Period Rate Per Second
DB cpu (seconds)			0.00	0.00	0.00	0.00
DB time (seconds)			4.58	6,473.70	0.15	30.54
db block changes			37,230.00	28,334.00	1,241.00	133.65
execute count			9,204.00	24,039,766.00	306.80	113,395.12
global cache cr block receive time (seconds)			0.00	0.00	0.00	0.00
global cache cr blocks received			0.00	0.00	0.00	0.00
global cache current block receive time (seconds)			0.00	0.00	0.00	0.00
global cache current blocks received			0.00	0.00	0.00	0.00
global cache get time (seconds)			0.00	0.00	0.00	0.00
global cache gets			0.00	0.00	0.00	0.00
opened cursors cumulative			9,214.00	24,051,822.00	307.13	113,451.99
parse count (total)			2,274.00	24,037,842.00	75.80	113,386.05
parse time cpu (seconds)			0.56	62.30	0.02	0.29
parse time elapsed (seconds)			0.28	1,900.38	0.01	8.96
physical reads			1,364.00	0.00	45.47	0.00
physical writes			354.00	74.00	11.80	0.35
redo size (KB)			11,738.20	8,102.63	391.27	38.22
session cursor cache hits			8,654.00	23,986,070.00	288.47	113,141.84
session logical reads			67,468.00	166,180.00	2,248.93	783.87
sql execute cpu time (seconds)			0.00	0.00	0.00	0.00
sql execute elapsed time (seconds)			0.00	0.00	0.00	0.00
user calls			292.00	44,128.00	9.73	208.15
user commits			0.00	24.00	0.00	0.11
user rollbacks			0.00	0.00	0.00	0.00
workarea executions - multipass			0.00	0.00	0.00	0.00
workarea executions - onepass			0.00	0.00	0.00	0.00
workarea executions - optimal			946.00	9,052.00	31.53	42.70

Previous 1-27 of 27 Next

Change Periods

The results show a significant improvement in performance. DB Time was reduced, indicating less overall workload on the system. Physical reads were eliminated, demonstrating reduced disk I/O due to improved query access paths.

Furthermore, parse time and redo size were reduced, and fewer user calls were required. These improvements indicate that queries are executed more efficiently, with reduced CPU usage and lower system overhead.

Key metrics monitored include:

- **CPU usage and wait events** – identifies high concurrency issues
- **Buffer Cache Hit Ratio (< 90%)** – indicates increased disk I/O
- **Log file sync waits** – highlights commit bottlenecks
- **Top SQL by elapsed time** – detects inefficient queries or plan regression

Special attention is given to materialized views, ensuring that:

- Queries utilize materialized views instead of base tables
- Execution plans remain stable
- Refresh operations do not introduce additional overhead

This combined approach ensures that performance improvements are maintained and that the system remains stable under varying workloads.